Fondazione ITS Academy per la Mobilità Sostenibile Aerospazio/Meccatronica e Servizi alle Imprese - Piemonte

Corso ITS 15ª Edizione

A.F. 2025-2026 - Torino

Tecnico Superiore per l'Automazione e la Robotica Industriale

Embedded System e AI per l'Automazione – EMBT03

Data: 16/03/2026 Rev: 00
File: C:\Users\Esame\Desktop\STM32-Workspace\LCD_DISPLAY\.pdf
Redatto da: Davide Sargiacomo, Roberto Andrea Marchetti.

INDICE

1 OBBIETTIVO.....3

2 COMPONENTI E STRUMENTI.....3

3 PROCEDIMENTO.....4

3.1 Studio della scheda di espansione e scelta del componente.....4
3.2 Studio del datasheet HTS221.....4
3.3 Schema di collegamento.....4
3.4 Configurazione in STM32CubeMX.....4
3.5 Implementazione del driver I2C bit-bang.....5
3.6 Implementazione del driver HTS221.....5
3.7 Implementazione del driver LCD e ciclo principale.....5

4 CONCLUSIONE.....6

Relazione Laboratorio

Esercitazione I2CBUS

1 Obbiettivo

L'obiettivo dell'esercitazione, assegnata nell'ambito del modulo EMBT03, è quello di interfacciare un componente I2C con la scheda STM32 NUCLEO-C031C6, creando le funzioni necessarie per inizializzarlo e visualizzare i dati acquisiti su un display LCD 16x2. Il componente scelto è il sensore capacitivo digitale di temperatura e umidità relativa HTS221, montato sulla scheda di espansione X-NUCLEO-IKS01A3. L'attività ha richiesto la lettura del datasheet, l'identificazione dell'indirizzo hardware I2C e della mappa interna dei registri, la configurazione del progetto in STM32CubeMX e la scrittura del codice per l'acquisizione e la visualizzazione dei dati. I valori misurati vengono mostrati in tempo reale sul display LCD, trasmessi via UART alla porta seriale del PC e visualizzati tramite il software PuTTY configurato a 115200 baud.

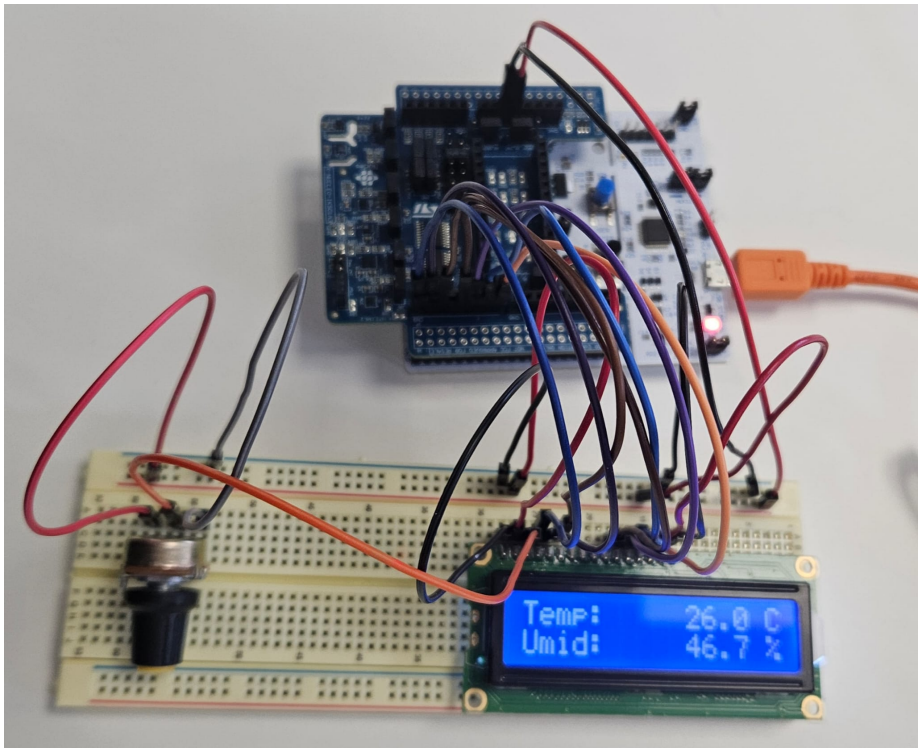
2 Componenti e Strumenti

Hardware:

- STM32 NUCLEO-C031C6
- X-NUCLEO-IKS01A3
- Sensore HTS221 per temperatura e umidità
- LCD 16x2 modello LCD-016N002M-TTI-ET

Software:

- STM32CubeIDE
- STM32CubeMX
- PuTTY



3 Procedimento

3.1 Studio della scheda di espansione e scelta del componente

Il primo passo è stato analizzare la scheda X-NUCLEO-IKS01A3 consultando il relativo datasheet per identificare i sensori disponibili. Tra i componenti presenti (LSM6DSO, LIS2MDL, LPS22HH, HTS221, STTS751) è stato scelto l'HTS221, sensore capacitivo digitale in grado di misurare sia la temperatura sia l'umidità relativa tramite un'unica interfaccia I2C.

3.2 Studio del datasheet HTS221

Il datasheet dell'HTS221 è stato analizzato seguendo la metodologia suggerita dal docente: recuperare le informazioni, leggere il datasheet, cercare esempi.

Dalla pagina 15 è stato identificato l'indirizzo hardware I2C del sensore: il bit SAD[6:0] è fisso a 1011111, che produce l'indirizzo 0xBE in scrittura (R/W = 0) e 0xBF in lettura (R/W = 1) nel formato a 8 bit utilizzato nel codice. Questo corrisponde alla convenzione I2C studiata in classe, per cui il byte di indirizzo è composto dai 7 bit dell'indirizzo del dispositivo più il bit R/W in posizione B0.

Dalla pagina 20 è stata identificata la mappa interna dei registri, individuando quelli principali: CTRL_REG1 per accendere e configurare il sensore, STATUS_REG per verificare la disponibilità dei dati, HUMIDITY_OUT_L/H e TEMP_OUT_L/H per leggere i valori grezzi di uscita, e i registri di calibrazione (H0_rH_x2, H1_rH_x2, T0_degC_x8, T1_degC_x8 con i relativi valori di riferimento ADC).

Dalla pagina 21 è stato compreso il significato dei registri: il sensore fornisce valori ADC a 16 bit con segno che non rappresentano direttamente la grandezza fisica, ma devono essere convertiti tramite interpolazione lineare usando i coefficienti di calibrazione memorizzati internamente. Il registro WHO_AM_I (indirizzo 0x0F) restituisce il valore fisso 0xBC e può essere usato per verificare la corretta comunicazione con il dispositivo.

3.3 Schema di collegamento

La scheda di espansione X-NUCLEO-IKS01A3 è stata collegata direttamente alla NUCLEO-C031C6 inserendo i pin maschi TH della scheda di espansione nei connettori femmina compatibili Arduino presenti sulla NUCLEO. Questo collegamento di tipo stack-on non richiede cavi aggiuntivi: le due schede si agganciano fisicamente l'una sull'altra e tutti i segnali necessari, comprese le linee I2C (SDA su PB9 e SCL su PB8), l'alimentazione a 3,3 V e il riferimento di massa, vengono trasferiti direttamente tramite i connettori. Il display LCD 16x2 è stato invece collegato separatamente ai pin GPIO della NUCLEO tramite cavi, con i pin dati e di controllo configurati come uscite push-pull per la gestione in modalità 4 bit.

3.4 Configurazione in STM32CubeMX

Il progetto è stato configurato tramite STM32CubeMX seguendo la procedura mostrata a lezione. I pin PB8 e PB9 sono stati configurati come uscite GPIO open-drain con pull-up interno attivato, condizione necessaria per rispettare la specifica del protocollo I2C: le linee open-drain permettono sia al master sia allo slave di controllare il bus senza conflitti, e il pull-up riporta la linea alta quando nessuno la forza bassa. Nella scheda GPIO Settings è stato verificato che entrambi i pin mostrassero la modalità open-drain. Nella scheda NVIC Settings è stato verificato che l'interrupt I2C non fosse abilitato, poiché la

comunicazione viene gestita interamente in modalità polling. Nella scheda Project Manager è stato verificato che le librerie generate includessero HAL per USART e LL per GPIO. Completata la configurazione è stato generato il codice base del progetto.

3.5 Implementazione del driver I2C bit-bang

Poiché la comunicazione I2C è gestita interamente via software, sono state implementate le funzioni di basso livello seguendo il modello di transazione studiato a lezione. La funzione I2C_Start() genera la condizione di START, ovvero SDA va basso mentre SCL è alto, oppure il Repeated START necessario per le operazioni di lettura. La funzione I2C_Stop() genera la condizione di STOP, ovvero SDA va alto mentre SCL è alto. La funzione I2C_WriteByte() invia un byte bit per bit dal più significativo e legge il bit ACK o NACK restituito dallo slave al nono impulso di clock. La funzione I2C_ReadByte() riceve un byte e invia ACK o NACK al termine in base al parametro passato.

La sequenza di scrittura segue il modello visto in aula: START, indirizzo dispositivo con R/W=0, numero del registro, valore da scrivere, STOP. La sequenza di lettura utilizza il Repeated START: START, indirizzo con R/W=0, numero del registro, Repeated START, indirizzo con R/W=1, lettura dei byte di dati con ACK su tutti tranne l'ultimo, NACK sull'ultimo byte, STOP.

3.6 Implementazione del driver HTS221

Sulle funzioni I2C di basso livello sono state costruite le funzioni specifiche per l'HTS221. La funzione HTS221_Init() attiva il sensore scrivendo 0x81 nel registro CTRL_REG1, impostando il bit PD=1 per accenderlo e ODR=01 per la frequenza di campionamento a 1 Hz, e legge i coefficienti di calibrazione dalla memoria interna del dispositivo. Per le letture multiple con auto-increment viene impostato il bit 7 dell'indirizzo del registro a 1, come indicato dal datasheet. Le funzioni HTS221_ReadTemperature() e HTS221_ReadHumidity() leggono i registri di output, ricostruiscono il valore a 16 bit con segno e applicano l'interpolazione lineare sui punti di calibrazione per ottenere rispettivamente la temperatura in gradi Celsius e l'umidità in percentuale. Il valore di umidità viene limitato all'intervallo fisicamente valido tra 0 e 100%.

3.7 Implementazione del driver LCD e ciclo principale

Il display LCD 16x2 è gestito in modalità 4 bit secondo lo standard HD44780. La funzione lcdInit() esegue la sequenza di inizializzazione prescritta dal datasheet del display con i ritardi necessari. Le funzioni lcdSendCmd() e lcdSendChar() inviano rispettivamente comandi e caratteri tramite due nibble consecutivi. Prima di ogni operazione lcdCheckBusy() verifica il flag di occupato del display impostando temporaneamente i pin dati come ingressi. La funzione lcdSetCursor() posiziona il cursore alla riga e colonna desiderate e lcdPrint() stampa una stringa di caratteri.

Nel ciclo principale, se il flag lcd_running è attivo, il programma legge temperatura e umidità ogni secondo, le formatta come stringhe e aggiorna entrambe le righe del display LCD, inviando contemporaneamente gli stessi valori via UART al PC. I dati trasmessi sono stati monitorati e verificati in tempo reale tramite PuTTY, configurato con una velocità di 115200 baud, 8 bit dati, nessuna parità e 1 bit di stop, che permetteva di leggere sul terminale le misure nel formato Temp: X.X C Umid: X.X% ad ogni aggiornamento. La pressione del pulsante utente su PC13, gestita tramite interrupt esterno EXTI con debounce software di 200 ms, inverte il flag lcd_running mettendo in pausa o riprendendo l'aggiornamento e commuta il LED interno come indicatore visivo.

4 Conclusione

Il progetto ha raggiunto il risultato richiesto dalla consegna: il sensore HTS221 è stato inizializzato correttamente e i valori di temperatura e umidità vengono visualizzati in tempo reale sul display LCD 16x2 e trasmessi via UART, con la possibilità di monitorare i dati anche sul terminale PuTTY. L'attività ha permesso di applicare concretamente i concetti del protocollo I2C trattati a lezione, dalla comprensione dell'indirizzamento slave con bit R/W, alle transazioni di scrittura e lettura con Repeated START, fino alla gestione dei bit ACK e NACK. Lo studio del datasheet dell'HTS221 ha evidenziato l'importanza di comprendere il significato dei registri interni e la necessità di applicare l'interpolazione lineare tramite i coefficienti di calibrazione per ottenere valori fisicamente corretti. La configurazione open-drain dei pin I2C e la verifica che l'interrupt non fosse abilitato hanno confermato l'importanza di una corretta impostazione hardware come prerequisito indispensabile per il funzionamento del software.